

A Guide to SSL 2.0: Security Flaws and Evolution to SSL 3.0

The importance of cybersecurity cannot be overstated. The SSL 2.0 handshake protocol plays a great role in ensuring data remains secure and does not reach the wrong hands. But it does have some weaknesses, and hence the emergence of SSL 3.0.



While current internet standards implement TLS 1.3 (Transport Layer Security), we've made significant strides in securing communication over the internet. The journey began with the development of SSL (Secure Sockets Layer) 1.0, which was never publicly released due to serious flaws. This was followed by SSL 2.0, which introduced a handshake protocol before establishing a connection between the client and server. Every protocol has its own vulnerabilities, and to this day, no single protocol provides a completely secure solution. However, in cybersecurity, we continuously strive to develop 'better' solutions, recognising that while these may not be perfect, they are the best options available at present.

The SSL 2.0 handshake protocol aims to send 'ClientHello' from the client and 'ServerHello' from the server over an insecure channel, typically over a Transmission Control Protocol (TCP) connection. The purpose of these messages is to establish the parameters for the secure session that will follow, including negotiating the encryption methods and exchanging the necessary cryptographic data. This includes a 'cipher suite' which is a list of encryption algorithms such as RC4-MD5, DES-CBC-MD5, and RC2-CBC-MD5 sent to the server to choose the most secure algorithm that both machines support, as shown in Figure 1.

The server responds with a 'ServerHello' message including the selected cipher suite, another random value (nonce), and the server's certificate

containing its public key. After these initial messages are exchanged, the process of key exchange and session establishment begins, and eventually, the communication is encrypted using the session key derived from the handshake process. This encryption secures subsequent data exchanges.

Key exchange methods

The generation of the PreMasterSecret (PMS) depends on the key exchange method being used during the SSL/TLS handshake. Figure 1 shows two of the typical methods.

RSA key exchange

- *Step 1: Client generates the PreMasterSecret:* The client randomly generates a 48-byte value known as the PreMasterSecret